

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 13

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

Goal **3**

1 Part 1: The numbers and the user side **3**

2 Part 2: The kernel side **9**

3 Part 3: A semaphore as a lock: mix **13**

Conclusion **16**

References **17**

Goal

In the two previous laboratory works semaphores were used, as they are offered by the C library of Linux. The purpose of this laboratory work is to build them instead, inside the xv6 kernel, as three new system calls: sem_init, sem_down and sem_up, numbered 24, 25 and 26. What the exercise shows is that a semaphore is not a complicated object: it is a number, a spinlock that protects that number, and the sleep and the wakeup of the kernel. A process that waits on it is not spinning and is not consuming any processor time; it is asleep, and the kernel wakes it when the number rises again.

The work has three parts. The first part gives the three calls their numbers and builds the user side, with the same recipe used for trace in an earlier laboratory work, repeated three times, plus the two test programs semtest and mix. The second part writes the kernel side: a new file kernel/sem.c with a table of eight semaphores shared by every process, a line in the list of objects of the Makefile so that the file is compiled into the kernel, and the entries of the three calls in the table of kernel/syscall.c. The third part uses one of these semaphores as a lock, in the program mix, where two processes print five lines each, one letter per write, first without any protection and then with the semaphore.

Part 1: The numbers and the user side

Three system calls are added at once, each of them following the same path from the name written by the programmer down to the register a7 that the kernel reads. This part performs six edits, after which the three numbers travel correctly but nothing in the kernel answers them yet.



Figure 1.1: The three new numbers in kernel/syscall.h

kernel/syscall.h – three lines were added at the end of the list: SYS_sem_init 24, SYS_sem_down 25 and SYS_sem_up 26. The number 22 remains sync and the number 23 remains trace, from the earlier laboratory work, which is why the new calls begin at 24; on a tree without trace the place 23 would simply stay empty in the table of the kernel, which does no harm, because the table is indexed by the number and not filled in order.

```
int sync(void);
int trace(int);
int sem_init(int, int);
int sem_down(int);
int sem_up(int);
```

Figure 1.2: The three declarations in user/user.h

user/user.h – the declarations int sem_init(int, int);, int sem_down(int); and int sem_up(int); were added after the ones of sync and trace. The first call takes two arguments, the number of the semaphore and its start value, while the other two take only the number. These are declarations and nothing more: they let a user program call the three functions by name, and say nothing about where the code of those functions is.

```
entry("sync");
entry("trace");
entry("sem_init");
entry("sem_down");
entry("sem_up");
```

Figure 1.3: The three entries in user/usys.pl

user/usys.pl – the three entries entry("sem_init");, entry("sem_down"); and entry("sem_up"); were added at the end of the script. At build time the script writes user/usys.S, where each entry becomes three instructions: the number of the call is placed into the register a7, the instruction ecall leaves the user mode and enters the kernel, and ret returns. The arguments themselves travel in the registers a0 and a1, and this is why the handler of sem_init must later read two of them and the handlers of the other two calls only one.

```
 GNU nano 8.7.1                    user/semtest.c *
// semtest.c: two tests for the kernel semaphores (Lab 12).
#include "kernel/types.h"
#include "user/user.h"
int
main(void)
{
// Test 1: order. Semaphore 0 starts at 0, so the child must wait.
sem_init(0, 0);
if (fork() == 0) {
sem_down(0);
```

Figure 1.4: The beginning of user/semtest.c in the editor

user/semtest.c – the test program contains two independent tests. The first one is about order: the semaphore 0 is initialized at 0, the process forks, and the child immediately calls sem_down(0), which must put it to sleep because the value is zero; the parent waits ten ticks with pause, prints its line and only then calls sem_up(0), which wakes the child. If the semaphores work, the line of the parent is printed first, although the child was

ready long before. The second test is a ping-pong between two processes, built from two semaphores that pass the turn to each other: the semaphore 1 starts at 1 and means that it is the turn of ping, the semaphore 2 starts at 0 and means that it is the turn of pong. Each of the two processes takes its own semaphore, prints its line and raises the semaphore of the other, so the two lines can only alternate. The source is given below.

```
// semtest.c: two tests for the kernel semaphores.
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  // Test 1: order. Semaphore 0 starts at 0, so the child must
   wait.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, semaphore 2 =
    pong's turn.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf("  pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

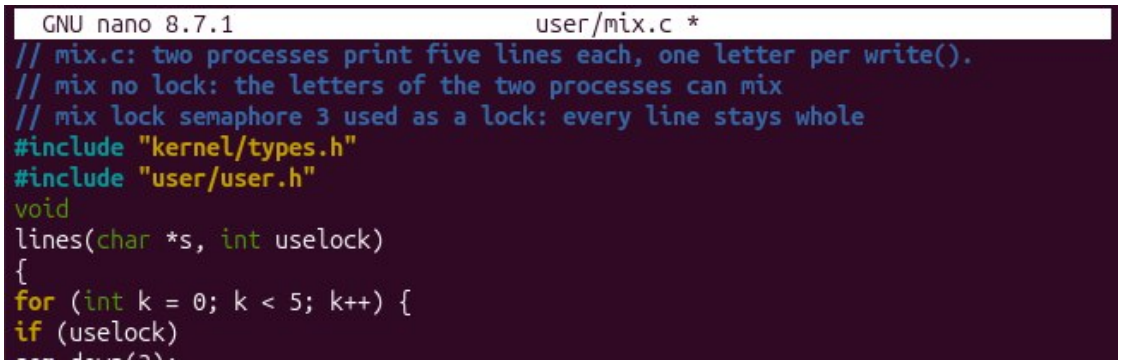


Figure 1.5: The beginning of user/mix.c in the editor

user/mix.c – the second program is needed in the third part and was created now, so that the Makefile receives both lines at once. It starts two processes that print five lines each, one letter per call to write, and it takes the lock only when it is given an argument: the test is argc $>$ 1, so the word lock is not read at all and any argument would switch the protection on. The semaphore 3 is initialized at 1 before the fork, which makes it an ordinary lock, as in the previous laboratory work where a POSIX semaphore started at 1 behaved exactly like a mutex. The source is given below.

```
1  // mix.c: two processes print five lines each, one letter per
       write().
2  //   mix        no lock: the letters of the two processes can
       mix
3  //   mix lock   semaphore 3 used as a lock: every line stays
       whole
4  #include "kernel/types.h"
5  #include "user/user.h"
6
7  void
8  lines(char *s, int uselock)
9  {
10   for (int k = 0; k < 5; k++) {
11     if (uselock)
12       sem_down(3);
13     for (char *p = s; *p; p++)
14       write(1, p, 1);
15     if (uselock)
16       sem_up(3);
17   }
18 }
19
20 int
21 main(int argc, char *argv[])
22 {
23   int uselock = (argc > 1);
24   sem_init(3, 1);
25   if (fork() == 0) {
26     lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
27     exit(0);
```

```
28    }
29    lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
30    wait(0);
31    exit(0);
32  }
```



Figure 1.6: The two programs added to UPROGS in the Makefile

Makefile – UPROGS – the lines $U/_semtest\ and $U/_mix\ were added after $U/_zombie\, each with a TAB at the beginning and with nothing after the backslash. The line $U/_trace\ of the earlier laboratory work is visible just below them and is untouched. Without these two lines the programs would be compiled by nothing and would never be copied into the image of the file system, and the shell of xv6 would answer exec semtest failed.

```
hart 1 starting
hart 2 starting
init: starting sh
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
```

Figure 1.7: Check 1: the numbers reach the kernel, which does not know them yet

Check 1 – $ semtest – the first lines are exactly the expected ones: 3 semtest: unknown sys call 24 and 4 semtest: unknown sys call 25, printed by the kernel with the pid of the process in front, the 3 being the parent and the 4 the child. The two messages prove that the user side is complete: the stubs placed the numbers 24 and 25 into the register a7, the instruction ecall entered the kernel, and the kernel received numbers for which its table holds nothing.

The second observation of this check is the important one. The line child: I ran after the parent is printed *before* the line parent: done, I let the child go, which is the opposite of what the program intends. Since sem_down(0) failed instead of blocking, nothing held the child back: it ran straight through, printed its line while the parent was still inside its ten ticks of pause, and its own text is a lie. This is exactly what the second part repairs,

and it is worth noticing that nothing crashed: a missing synchronization does not produce an error, only a wrong order.

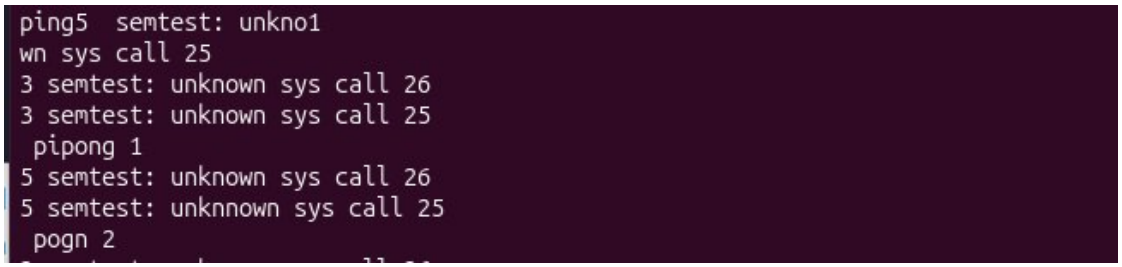


Figure 1.8: The continuation of Check 1: the error lines and the two processes mix together

Check 1 – the ping-pong without semaphores – further down the same screen the output becomes scrambled: pipong 1, pogn 2, sentest, unknno wn sys call 25. Nothing is broken here either, and the explanation is the subject of the third part. Two processes print at the same time, and the printf of xv6 writes one character per call to write, so the letters of ping and of pong arrive at the console interleaved: pipong is literally the words ping and pong written one character at a time into each other. The messages of the kernel, printed with printk, enter the same stream and are cut in the same way, which is why unknown appears as unknno wn. Without the semaphores there is nothing to keep the turn, nothing to keep a line whole, and the result is the mixture seen in the figure. The six edits of this part are summarized in Table 1.

File	**What was added**	**Why it is needed**
kernel/syscall.h	the numbers 24, 25 and 26	names the three calls on both sides
user/user.h	the three declarations	lets a user program call them by name
user/usys.pl	the three entries	generates the stubs: li a7, N; ecall; ret
user/semtest.c	the test of order and the ping-pong	proves that a process really sleeps
user/mix.c	two processes, one letter per write	shows the semaphore used as a lock
Makefile	$U/_semtest\ and $U/_mix\	compiles both and puts them on the disk

Table 1: The six edits of the user side

Table 1 shows that none of these files contains any logic of the semaphores: they only carry three numbers to the register a7 and provide the two programs that will test what the kernel does with them.

Part 2: The kernel side

The three numbers now arrive and are refused. This part writes what happens when they arrive, and it is here that the semaphore is actually built, out of three ingredients that the kernel already had: a number, a spinlock and the pair formed by sleep and wakeup.



Figure 2.1: The beginning of the new file kernel/sem.c

kernel/sem.c – the new file holds a static table of eight semaphores, shared by every process of the system, each of them a structure with a spinlock and an integer. A spinlock filled with zeros is a free spinlock, which is why the table needs no initialization at boot. The three handlers are short. sys_sem_init checks that the number is inside the table and that the value is not negative, then takes the spinlock, writes the value and releases it. sys_sem_up takes the spinlock, adds one to the value, calls wakeup on the address of the semaphore and releases the spinlock. sys_sem_down is the heart of the file and is explained after the listing. The source is given below.

```
1  // sem.c: counting semaphores inside the xv6 kernel.
2  // A small table of NSEM semaphores, shared by every process.
3  // Each one is a number plus a spinlock that protects the number
       .
4  #include "types.h"
5  #include "riscv.h"
6  #include "defs.h"
7  #include "param.h"
8  #include "spinlock.h"
9  #include "proc.h"
10
11 #define NSEM 8
12
13 struct sem {
14   struct spinlock lock;    // a zeroed spinlock is a free
       spinlock
15   int value;               // how many may still pass
16 };
17
18 static struct sem sems[NSEM];
19
```

```
20 // sem_init(id, value): give semaphore id its start value.
21 uint64
22 sys_sem_init(void)
23 {
24   int id, value;
25   argint(0, &id);
26   argint(1, &value);
27   if (id < 0 || id >= NSEM || value < 0)
28     return -1;
29   acquire(&sems[id].lock);
30   sems[id].value = value;
31   release(&sems[id].lock);
32   return 0;
33 }
34
35 // sem_down(id): take one unit; if there is none, sleep until
       there is.
36 uint64
37 sys_sem_down(void)
38 {
39   int id;
40   argint(0, &id);
41   if (id < 0 || id >= NSEM)
42     return -1;
43   struct sem *s = &sems[id];
44
45   acquire(&s->lock);
46   while (s->value == 0) {
47     sleep_prepare(s);      // I will sleep on the address of s
48     release(&s->lock);     // never sleep holding a spinlock
49     sleep();               // returns at once if sem_up came in
      between
50     acquire(&s->lock);
51     if (killed(myproc())) {   // kill must still work on a
      waiting process
52       release(&s->lock);
53       return -1;
54     }
55   }
56   s->value--;
57   release(&s->lock);
58   return 0;
59 }
60
61 // sem_up(id): give one unit back and wake whoever sleeps on s.
62 uint64
63 sys_sem_up(void)
64 {
65   int id;
66   argint(0, &id);
67   if (id < 0 || id >= NSEM)
```

```
68      return -1;
69    struct sem *s = &sems[id];
70
71    acquire(&s->lock);
72    s->value++;
73    wakeup(s);
74    release(&s->lock);
75    return 0;
76  }
```

The order of the four lines inside sem_down – the loop is the whole difficulty of the file, and each of its four lines is placed where it is for a reason. The process first registers the address of the semaphore as the thing it is going to wait for, with sleep_prepare(s), while it still holds the spinlock; only then does it release the spinlock; only then does it actually go to sleep; and when it wakes it takes the spinlock again before looking at the value. Registering before releasing is what closes the window in which a sem_up could arrive between the release and the sleep: if that happens, the wakeup finds the process already registered and sleep() returns at once instead of blocking for ever. This defect has a name, the lost wakeup, and the two-step sleep of this kernel exists precisely to prevent it. The test killed(myproc()) after waking is what keeps the command kill working on a process that is waiting on a semaphore, which would otherwise be unkillable.

```
$K/syscall.o \
$K/sysproc.o \
$K/sem.o \
```

Figure 2.2: The object file added to the OBJS list of the Makefile

Makefile – OBJS – the line $K/sem.o \ was added after $K/sysproc.o \, with two spaces at the beginning this time and not a TAB, because this list has a different indentation from UPROGS. Without it the new file would simply not be compiled, and the build would stop with the message undefined reference to sys_sem_init, since the table of the next step names a function that nothing provides.

```
extern uint64 sys_sync(void);
extern uint64 sys_trace(void);
extern uint64 sys_sem_init(void);
extern uint64 sys_sem_down(void);
```

Figure 2.3: The declarations of the three handlers in kernel/syscall.c

kernel/syscall.c – extern – the three lines extern uint64 sys_sem_init(void);, extern uint64 sys_sem_down(void); and extern uint64 sys_sem_up(void); were added after the ones of sys_sync and sys_trace. The handlers live in another file, so these lines tell the compiler that such functions exist and that the linker will find them; without them the names used in the table below would be unknown and the file would not compile.



Figure 2.4: The three entries of the new calls in the table of the kernel

kernel/syscall.c – the table – the three lines [SYS_sem_init] = sys_sem_init,, [SYS_sem_down] = sys_sem_down, and [SYS_sem_up] = sys_sem_up, were added to the array syscalls, right after the entry of trace. These are the lines that make the messages of Check 1 disappear, because the places 24, 25 and 26 of the array are no longer empty. The notation with the name of the constant inside brackets places each element at its own index, so the order of the lines in the source does not matter and the gap at the place 23 on a tree without trace would be harmless.

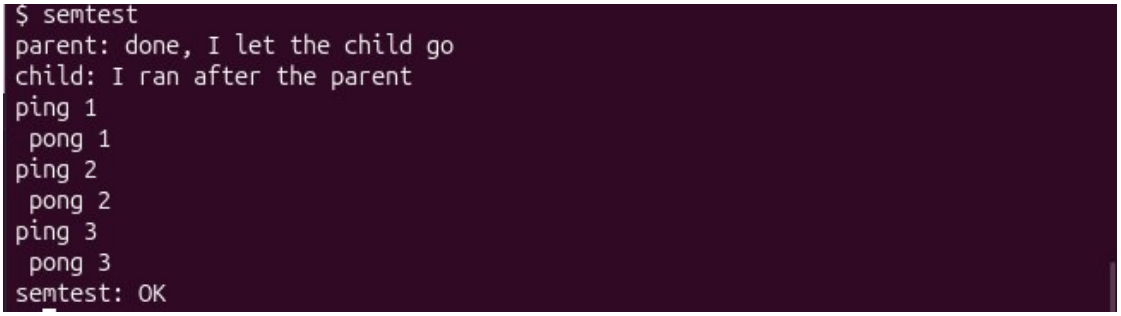


Figure 2.5: Check 2: the order is respected and ping and pong alternate

Check 2 – $ semtest – after Ctrl-A X and a new make qemu the test passes completely. The line parent: done, I let the child go is now printed first and child: I ran after the parent second, which is the reverse of Check 1 and the proof that the child really slept: it was ready immediately, it called sem_down on a semaphore whose value was 0, and it stayed asleep for the whole ten ticks of the pause of the parent, consuming no processor time, until sem_up woke it. Below that, ping and pong alternate strictly three times each, never two of the same in a row, because each process holds its own semaphore and can only advance when the other one raises it. The last line is semtest: OK. The edits of the kernel side are summarized in Table 2, and the three ingredients of the semaphore in Table 3.

File	**What was added**	**What it does**
kernel/sem.c	the whole file: the table and the three handlers	the semaphores themselves
Makefile, OBJS	$K/sem.o \	compiles the new file into the kernel
kernel/syscall.c	three extern declarations	announces the handlers of the other file
kernel/syscall.c	three entries in the table syscalls	fills the places 24, 25 and 26

Table 2: The edits of the kernel side

Ingredient	In the code	What it provides
a number	int value	how many processes may still pass
a spinlock	struct spinlock lock	the number is read and changed by one core at a time
sleep and wakeup	sleep_prepare, sleep, wakeup	a waiting process uses no processor at all

Table 3: The three ingredients a semaphore is made of

Table 3 is the lesson of this part: nothing new had to be invented. The kernel already protected its own data with spinlocks and already knew how to put a process to sleep and wake it; the semaphore is only those pieces put together around one integer.

Part 3: A semaphore as a lock: mix

Questions:

- Without the lock, why do the letters mix? How many system calls does one line cost?
- With the lock every line is whole. Which two lines of mix.c make one line a critical section?
- In sem_down the test is while (s->value == 0), not if. Why can a process wake up and still find 0?
- sem_down calls release(&s->lock) before sleep(). What would go wrong if it slept holding the spinlock?

Answers:

- Because one line is not one system call but thirty-one. The inner loop writes a single byte at a time, with write(1, p, 1) for every character, and the string is thirty letters plus the newline, so a line costs 31 calls and a whole process 155 of them. Each individual write is indivisible, since the console driver takes its own lock for one character, but nothing holds the console across the thirty-one calls of a line. Between any two of them the other process can be scheduled, and since make qemu starts xv6 with three processors the two processes are in fact running at the same time on different cores, so the characters of one arrive in the middle of the line of the other. It is worth adding that the printf of xv6 is not buffered either, unlike the one of Linux: it also writes one character per system call, which is why even the error messages of Check 1 came out cut into pieces.
- The two calls that surround the inner loop: if (uselock) sem_down(3); before it and if (uselock) sem_up(3); after it. The semaphore 3 is initialized at 1 by sem_init(3, 1), which makes it a lock, exactly as a POSIX semaphore started at 1 behaved like a mutex in the previous laboratory work: the first process takes the single unit, the value falls to 0, and the second one sleeps inside sem_down until the first calls sem_up. The critical section is therefore the whole group of thirty-one writes, and

a line can no longer be interrupted in the middle.

- Because waking up is not the same as obtaining a unit. The call wakeup(s) of this kernel wakes every process that sleeps on that address, not one of them, so if several processes wait and a single sem_up adds one unit, all of them become runnable, one takes the unit and the others must find the value at 0 again. Even with a single waiter the same thing can happen, because a woken process must take the spinlock back before it can look at the value, and in that interval a third process that never slept can call sem_down and take the unit first. With if instead of while the loser would fall through to s->value–, which would drive the value to −1 and let two processes into a critical section that admits one. This is the same rule as the while around pthread_cond_wait in the previous laboratory work: a condition must always be tested again after waking.
- The system would deadlock, and in this kernel it would most probably panic first. A spinlock is not released by falling asleep: the sleeping process keeps owning it. Another process that calls sem_up would then reach acquire(&s->lock) and spin there for ever, burning its core without ever reaching s->value++ or wakeup(s), so the only thing that could free the sleeper is the very thing that the sleeper is blocking. Moreover acquire disables the interrupts on that core, and the scheduler of xv6 refuses to switch out a process that still holds a spinlock, which ends in a panic about the locks rather than in a silent freeze. The comment in the file says it in four words: never sleep holding a spinlock.

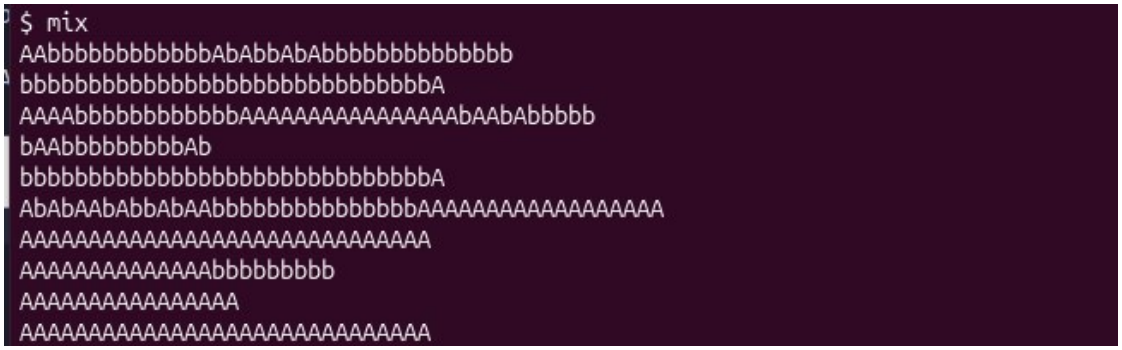


Figure 3.1: Without the lock: the letters of the two processes interleave

$ mix – with no argument the test argc > 1 is false, so neither sem_down nor sem_up is called and the two processes write freely. The result is the expected mixture: lines that begin with capital letters and continue with small ones, lines of every possible length, and a few lines that happen to come out whole because the other process was busy elsewhere at that moment. No character is ever lost and none is doubled, because every single write is indivisible; what is lost is the grouping, that is, the fact that thirty-one consecutive characters belong together. The ten lines of the output are still ten newlines, but they fall in the wrong places.

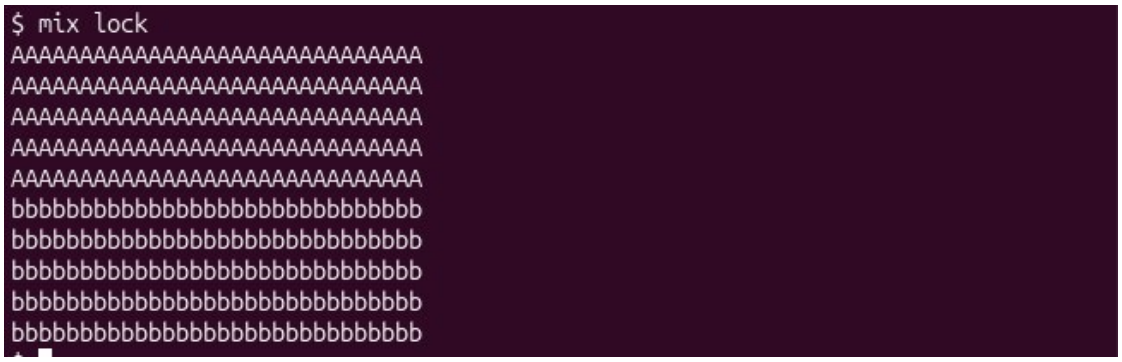


Figure 3.2: With the lock: every line is whole

$ mix lock – with an argument the semaphore is used, and the output is perfectly clean: five lines of thirty capital letters, then five lines of thirty small letters, with no mixture anywhere. The grouping of the five lines of each process one after the other is not guaranteed by anything and is only what happened in this run: a semaphore gives no fairness, so the process that has just released the lock is often the one that takes it again immediately, because it is still running on its core while the other one has to be woken up. An output in which the two kinds of lines alternated would be equally correct; what the semaphore guarantees is that no line is ever broken, and that guarantee is visible in every line of the figure. The two runs are compared in Table 4.

Command	**Semaphore 3**	**One line is**	**Result on the screen**
mix	not used	31 separate system calls	the letters of the two processes mix
mix lock	used, starts at 1	one critical section	every line is whole, 30 letters

Table 4: The same program, with and without the lock

Table 4 shows that the program itself was not changed between the two runs: the same five lines, the same thirty-one writes each, the same two processes. The only difference is whether the group of writes is declared indivisible, and that declaration is the semaphore built in the second part.

Conclusion

The laboratory work built inside the xv6 kernel the tool that two earlier works had only used from the C library of Linux. In the first part the three calls received the numbers 24, 25 and 26 and the user side was completed with the same recipe as the call trace, repeated three times: the constants in kernel/syscall.h, the declarations in user/user.h, the entries in user/usys.pl that generate the stubs with the instruction ecall, the two test programs and their lines in UPROGS. The check at the end of that part printed unknown sys call 24 and 25, which proved that the numbers travelled correctly, and it showed two defects that the absence of the semaphores causes: the child printed its line before the parent, although its text claims the opposite, and the ping-pong came out scrambled, with words such as pipong, because two processes were writing one character at a time into the same console with nothing to keep the turn.

In the second part the kernel side was written. The new file kernel/sem.c holds a table of eight semaphores shared by every process, each of them a number protected by a spinlock, and three short handlers; the Makefile received the object file in the list OBJS, and kernel/syscall.c received three extern declarations and three entries in the table of the calls. The function sem_down is where the whole difficulty lies, and the order of its four lines is what makes it correct: the process registers the address of the semaphore while it still holds the spinlock, releases the spinlock, sleeps, and takes the spinlock again on waking. Registering before releasing closes the window of the lost wakeup, and releasing before sleeping avoids the deadlock that would follow from sleeping with a spinlock in hand. The check then showed the parent printing first and the child second, ping and pong alternating strictly three times each, and the line semtest: OK.

In the third part one of those semaphores was used as a lock. Without it, the two processes of mix mixed their letters, because a line of thirty characters and a newline costs thirty-one separate system calls and nothing held them together, while xv6 ran on three processors and the printf of its library writes one character at a time. With the semaphore, initialized at 1 and taken around the inner loop, every line came out whole. The two remaining questions concern the kernel code itself: the test of the value must be a while and not an if, because wakeup wakes all the waiters and a unit can be taken by another process before a woken one gets the spinlock back; and the spinlock must be released before sleeping, because a spinlock is not freed by falling asleep and the process that would wake the sleeper would spin on it for ever. Overall, the work showed that a semaphore is only a number, a spinlock and the sleep and wakeup of the kernel, and that a process waiting on it is genuinely asleep and consumes no processor time; all the objectives of the laboratory work were achieved.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*
   `https://github.com/gabriela-pricop/os-2026-GabrielaPricop`
2. *Xv6, a simple Unix-like teaching operating system: the book and the source code (MIT 6.1810)*
   `https://pdos.csail.mit.edu/6.828/xv6`
3. *xv6: a simple, Unix-like teaching operating system, the book: locking, and sleep and wakeup*
   `https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf`
4. *sem_ overview(7) – overview of POSIX semaphores, for comparison with the ones built here*
   `https://man7.org/linux/man-pages/man7/sem_overview.7.html`